

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
INSTITUTO DE INFORMÁTICA

Pedro Strauch Fleury
Danilo Vieira Melo dos Anjos
Miguel Predebon Abichequer
Felipe Caetano Arellano

Trabalho Final da Cadeira de Desenvolvimento de Software

Prof. Dr. Marcelo Soares Pimenta

Porto Alegre
2026

1- Dependências

Essa seção se resume a especificar as dependências para rodar o projeto:

Linguagem: Python 3.12.10

Bibliotecas: Mido 1.3
Pygame 2.6.1
Tkinter

Outros: Pasta chamada “Saidas” para salvar arquivos midi exportados

2- Introdução e Requisitos

2.1 - Introdução

Este relatório se refere ao progresso realizado pelo nosso grupo (Grupo 3) em completar o trabalho do conversor de texto em música, durante a segunda etapa do trabalho final da disciplina de Desenvolvimento de Software do Prof. Dr. Marcelo Soares Pimenta. O objetivo central do projeto é desenvolver um software que atenda às boas práticas e métodos de programação apresentados em aula, sendo ele capaz de transformar um texto em uma música composta de múltiplas melodias síncronas (inspirado pelas fugas de Bach) segundo o mapeamento de notas e operações fornecido.

Visando a viabilizar a implementação técnica dos requisitos do trabalho, algumas premissas serão assumidas:

Premissa 1 - O software será capaz de editar e utilizar apenas arquivos codificados usando UTF-8 com a extensão .txt.

Premissa 2 - Os arquivos MIDI a serem salvos ficarão num diretório padrão (“/Saídas”) e também seguirão um padrão de nomeação.

Premissa 3 - O usuário pode especificar todos os parâmetros iniciais da música (atraso, bpm, volume e instrumento), mas caso ele não o faça, os parâmetros que dependem de linha (todos exceto bpm) serão iniciados de forma padronizada.

2.2 - Requisitos

Com base nas premissas, planejamos implementar nosso software de forma a cumprir com requisitos especificados no enunciado da fase 2:

Requisitos funcionais:

- Capacidade do usuário de ler, criar, editar e transformar em música arquivos de texto;
- É possível utilizar todos os 128 instrumentos do padrão General MIDI;
- Capacidade de exportar o áudio gerado como arquivo MIDI;
- Conseguir tocar o áudio gerado;
- Ajuste de parâmetros iniciais do áudio (bpm, instrumento, atraso e volume).

Requisitos não-funcionais:

- Se durante a execução do programa ocorrer um erro, o usuário deve ser comunicado de forma clara sobre o erro;
- O software deve ser intuitivo e de fácil utilização;
- O software deve ser orientado a objetos.

3 - Implementação prática

3.1 - Classes

Classe interface

Atributos:

botões # nativos da biblioteca Tkinter, os tk.Button são instanciados para cobrir cada uma das possíveis ações do usuário, por exemplo: definir bpm, tocar, gerar midi.

geometry # dimensões da interface.

bpm # o bpm selecionado.

listaDeVolumes # volume de cada uma das vozes (linhas).

listaDeInstrumentos # o instrumento selecionado para cada uma das vozes.

Métodos:

procurarArquivos # abre uma caixa de diálogo que permite ao usuário selecionar um arquivo do armazenamento local.

input_e_numero # trata exceções e verifica se a entrada fornecida pelo usuário é de um tipo conhecido pelo programa.

chama_gerarMIDI # chama a função transcrever() com os parâmetros e a entrada definidos anteriormente pelo usuário. Retorna o arquivo MIDI gerado com os dados fornecidos.

Classe inputLista

entradas # nativas da biblioteca Tkinter, as instâncias de tk.Entry permitem ao usuário fornecer entradas para o programa.

botões # relacionados com a seleção da linha desejada para receber a entrada.

Métodos:

Funções que tratam exceções de entrada, garantindo que ela pertença ao conjunto previsto pela especificação.

Classe Nota

Atributos:

frequencia #string de 2 chars que representa a nota a ser tocada e sua oitava

bpm #int que representa o inverso do tempo a se esperar entre essa nota e a próxima

volume #int que guarda a intensidade da nota

instrumento #int que diz com qual instrumento a nota será tocada

Métodos:

print() #printa a frequência da nota (usado apenas em testes)

Classe Interpretador

Atributos:

partitura #matriz de Notas

Métodos:

transcrever(arq, bpm, inst[], vol[], oitava[]) #gera a matriz de Notas a partir do .txt e dos parâmetros iniciais.

iniciar_parametros(linhas,instrumentos,volumes)# método privado, inicializa dados necessários para o método transcrever.

valor_invalido(valor) #Método privado, testa se os valores obedecem às restrições impostas internamente pelas tecnologias.

Classe Música

Atributos:

estado #pode ser “não iniciada”, “pausada” ou “tocando”

posicao #última parte da música a ser tocada

mensagens #matriz de mensagens MIDI (objeto python da biblioteca MIDO)

Métodos:

iniciar(interpretador) #recebe um objeto da classe interpretador para criar as mensagens MIDI da música.

tocar() #toca a música, sendo que é preciso que a música tenha sido iniciada antes.

parar() #para de tocar a música e reinicia posição tempo para zero.

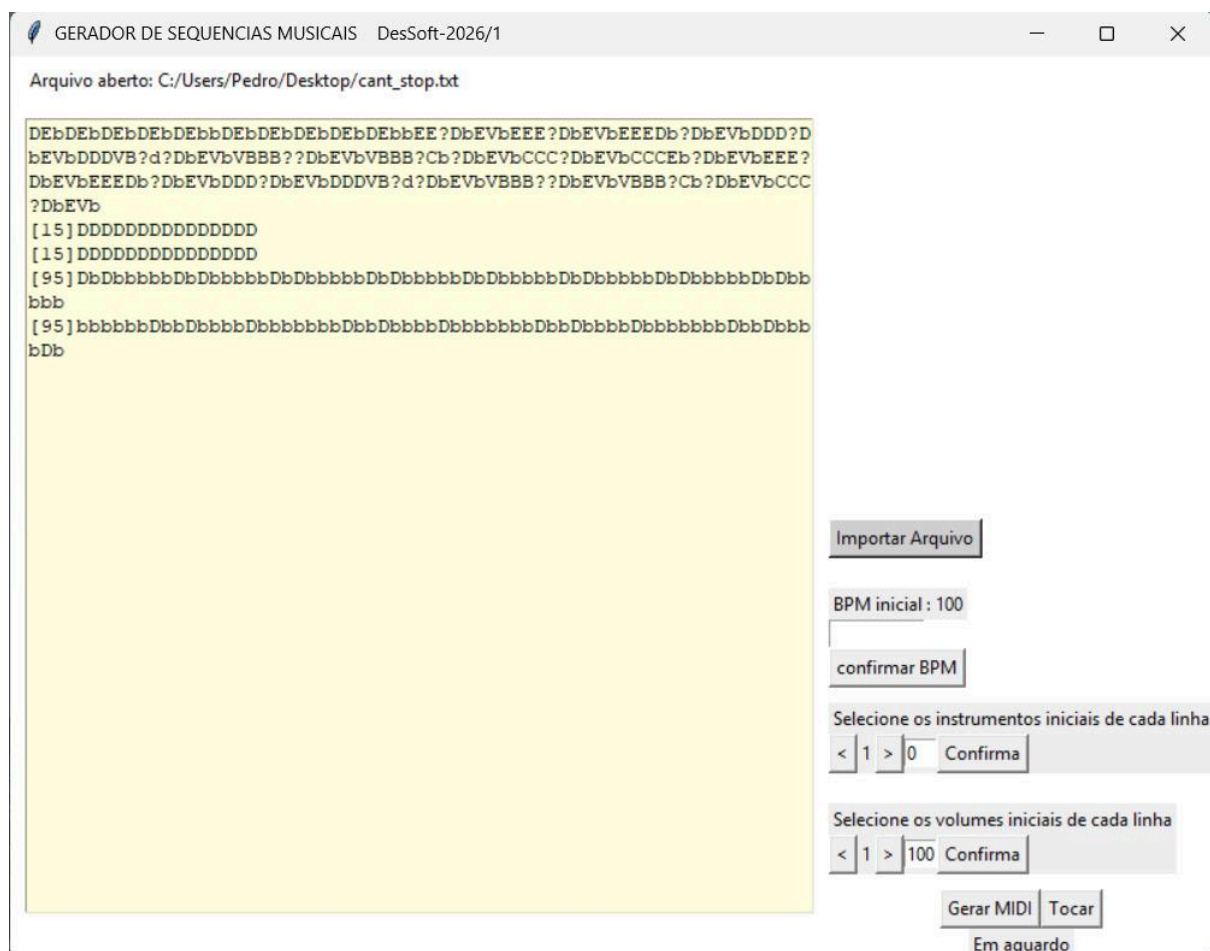
3.2 Casos de Uso

Quando o usuário inicia o programa ele tem duas opções: escrever um texto ou importar um arquivo - que pode ser editado. O usuário pode, então, decidir o volume e instrumento iniciais de cada linha ou deixar o padrão. Essas informações são passadas para o interpretador que transforma o bloco de texto e as escolhas do usuário em um formato padrão: uma matriz de objetos da classe Nota chamada de partitura. A classe Musica usa essa partitura para gerar um arquivo Midi com auxílio da biblioteca Mido, que torna possível tratar mensagens midi como objetos python. A classe interface, com auxílio da biblioteca PyGame, toca o arquivo Midi gerado.

4 - Testes

Nossos testes foram feitos de forma unitária, testando cada método separadamente. Posteriormente, realizamos testes de integração para garantir que o todo funcionaria sem problema. O teste da classe interpretador consistiu em chamar o interpretador, e printar cada nota, para garantir que as especificações foram seguidas. O da classe música consistiu em criar arquivos midi usando o formato intermediário, gerado pelo interpretador, que já sabíamos estar correto devido ao teste anterior. Nessa parte, alguns problemas mais complexos foram encontrados, sendo preciso printar os objetos gerados pela biblioteca Mido para identificar precisamente o problema. Para testar a interface, utilizamos o teste de aceitação, visando a avaliar sua usabilidade. Assim, tomamos o lugar do usuário ao utilizar o software, validando as diferentes funcionalidades por ele oferecidas. Os testes finais foram feitos no sistema como um todo, em busca de comportamentos inesperados. Todas as seções foram feitas com diversas entradas de texto diferentes, que se encontram na pasta testes de nosso repositório.

5 - Descrição da Interface



1: Campo de Texto:

Campo da interface em que o usuário pode digitar e editar manualmente a entrada de texto para gerar a sequência musical final, ou em que o conteúdo do arquivo .txt importado é exibido. Texto inserido por importação de arquivos pode ser mudado normalmente.

2: Botão Importar Arquivo:

Ao ser selecionado com o mouse, abre um diálogo que permite ao usuário navegar pelos arquivos de seu computador, para que ele assim procure e selecione um arquivo do tipo .txt caso deseje utilizá-lo como fonte para o input textual, que será enviado para o programa.

3: Campo de Seleção de BPM:

Permite que o usuário mude o BPM inicial da música gerada. A área de texto pode receber um número inteiro entre 0 e 400 e o botão confirma fixa a escolha do usuário.

4: Campos de Seleção de Instrumentos e Volumes:

As áreas de seleção de instrumentos e volumes apresentam comportamentos similares entre si. O usuário pode utilizar as setas ao lado esquerdo para selecionar qual linha receberá o novo parâmetro, o campo de texto permite que um número de 0 a 127 (valor para o volume ou número correspondente a um instrumento MIDI) seja selecionado, e o botão confirma a escolha do usuário caso o número inserido seja válido. Por padrão, os instrumentos iniciais de cada linha são MIDI 0 (piano), e os volumes iniciais de cada linha 100.

5: Botões Gerar MIDI e Tocar:

O botão “Gerar MIDI” criará um arquivo “MIDI.mid” na pasta Saídas, caso a entrada fornecida pelo usuário seja válida. O botão “Tocar” acessa esse mesmo arquivo e inicia o processo de reproduzir o seu conteúdo. Caso não exista um arquivo MIDI.mid na pasta Saídas, a função de gerar midi criará um novo. Caso já exista um arquivo MIDI.mid na pasta, ele pode ser sobrescrito em execuções subsequentes.

6 - As Revisões, conforme apontadas pelo Grupo Revisor

6.1 Introdução aos Problemas apontados

Na noite da entrega da Fase 3, o grupo Revisor entrou em contato conosco através do e-mail de um dos nossos integrantes para enviar-nos seu relatório. Nele havia elogios e também muitos pontos importantes para melhorarmos nosso software:

- No arquivo *classe_interface.py*, os métodos *chama_tocar* e *chama_gerarMIDI* utilizavam a mesma condição inicial para executarem uma ação.

- A classe *InterfaceGrafica* desempenha múltiplas responsabilidades.
- O método transcrever da classe *Interpretador* era longo em tamanho e nele havia um switch case extenso.
- A nomenclatura não estava plenamente padronizada, com variáveis e métodos seguindo modelos diferentes de um arquivo para outro: “camelCase”, “snake_case” e “PascalCase”.
- Comentários de código morto e explicações da função em si, em vez de explicar porquê foi implementada daquela forma.
- Tratamento para exceções na interação com o usuário, envolvendo input/output.

6.2 Soluções e Reconhecimentos dos Problemas

Após a leitura do material do Grupo Revisor, combinada com uma autocrítica oriunda de nós mesmos, conseguimos chegar a estas resoluções:

- Na *classeinterface.py*, a *chama_tocar* espera pela condicional do arquivo .mid ter sido criado ou não, enquanto a *chama_gerarMIDI* testa se o número de linhas está dentro das 16 propostas para criar o arquivo .mid a partir do texto.
- Devido ao jeito que a biblioteca de interface gráfica funciona, a separação entre interface gráfica e lógica de negócio da classe fica complexa de fatorar. Como a GUI não fazia parte do domínio problema, acabamos por refatorá-la parcialmente, embora ainda existam pontos a melhorar.
- No método transcrever, os parâmetros iniciais de cada voz foram separados e tratados por outro método, que também lida com valores inválidos no input por texto, ou por digitação.
- Uma uniformidade na nomenclatura foi alcançada, ao adotarmos a forma “snake_case” para nomear métodos e atributos, e “PascalCase” para classes.
- Os comentários referentes a “código morto” foram removidos, pois eram funcionalidades parciais destinadas a testar a interface antes dela estar totalmente concretizada, e os métodos comentados foram refatorados, nos quais seus comentários não eram mais necessários.
- Foram adicionadas condicionais na classe *InterfaceGrafica*, que cuidam de casos da BPM inicial inserida estar incorreta, erro na geração do arquivo MIDI, falta de arquivo MIDI para ser tocado pelo executável, e número de linhas acima do previsto.

As observações do Grupo Revisor foram de grande ajuda para o aumento da qualidade do software, resultando, por fim, em uma implementação mais limpa e legível.

